

Deep dive into the murky waters of script loading



Jake Archibald



(<https://twitter.com/jaffathecake>)



(<https://jakearchibald.com/>)

Introduction

In this article I'm going to teach you how to load some JavaScript in the browser and execute it.

No, wait, come back! I know it sounds mundane and simple, but remember, this is happening in the browser where the theoretically simple becomes a legacy-driven quirk-hole. Knowing these quirks lets you pick the fastest, least disruptive way to load scripts. If you're on a tight schedule, skip to the quick reference.

For starters, here's how the spec (<http://www.whatwg.org/specs/web-apps/current-work/multipage/scripting-1.html#script>) defines the various ways a script could download and execute:





The WHATWG on script loading

Like all of the WHATWG specs, it initially looks like the aftermath of a cluster bomb in a scrabble factory, but once you’ve read it for the 5th time and wiped the blood from your eyes, it’s actually pretty interesting:

My first script include

```
<script src="//other-domain.com/1.js"></script>
<script src="2.js"></script>
```

Ahh, blissful simplicity. Here the browser will download both scripts in parallel and execute them as soon as possible, maintaining their order. “2.js” won’t execute until “1.js” has executed (or failed to do so), “1.js” won’t execute until the previous script or stylesheet has

executed, etc etc.

Unfortunately, the browser blocks further rendering of the page while all this is happening. This is due to DOM APIs from “the first age of the web” that allow strings to be appended onto the content the parser is chewing through, such as `document.write`. Newer browsers will continue to scan or parse the document in the background and trigger downloads for external content it may need (js, images, css etc), but rendering is still blocked.

This is why the great and the good of the performance world recommend putting script elements at the end of your document, as it blocks as little content as possible. Unfortunately it means your script isn’t seen by the browser until it downloads all your HTML, and by that point it’s started downloading other content, such as CSS, images and iframes. Modern browsers are smart enough to give priority to JavaScript over imagery, but we can do better.

Thanks IE! (no, I’m not being sarcastic)

```
<script src="//other-domain.com/1.js" defer></script>
<script src="2.js" defer></script>
```

Microsoft recognised these performance issues and introduced “defer” into Internet Explorer 4. This basically says “I promise not to inject stuff into the parser using things like `document.write`. If I break that promise, you are free to punish me in any way you see fit”. This attribute made it into HTML4 (<http://www.w3.org/TR/html401/interact/scripts.html#h-18.2.1>) and appeared in other browsers.

In the above example, the browser will download both scripts in parallel and execute them just before `DOMContentLoaded` fires,

maintaining their order.

Like a cluster-bomb in a sheep factory, “defer” became a wooly mess. Between “src” and “defer” attributes, and script tags vs dynamically added scripts, we have 6 patterns of adding a script. Of course, the browsers didn’t agree on the order they should execute. Mozilla wrote a great piece on the problem (<https://hacks.mozilla.org/2009/06/defer/>) as it stood back in 2009.

The WHATWG made the behaviour explicit, declaring “defer” to have no effect on scripts that were dynamically added, or lacked “src”. Otherwise, deferred scripts should run after the document had parsed, in the order they were added.

Thanks IE! (ok, now I’m being sarcastic)

It giveth, it taketh away. Unfortunately there’s a nasty bug in IE4-9 that can cause scripts to execute in an unexpected order (<https://github.com/h5bp/lazyweb-requests/issues/42>). Here’s what happens:

1.js

```
console.log('1');  
document.getElementsByTagName('p')[0].innerHTML = 'Changing some content';  
console.log('2');
```

2.js

```
console.log('3');
```

Assuming there's a paragraph on the page, the expected order of logs is [1, 2, 3], although in IE9 and below you get [1, 3, 2]. Particular DOM operations cause IE to pause current script execution and execute other pending scripts before continuing.

However, even in non-buggy implementations, such as IE10 and other browsers, script execution is delayed until the whole document has downloaded and parsed. This can be convenient if you're going to wait for `DOMContentLoaded` anyway, but if you want to be really aggressive with performance, you can start adding listeners and bootstrapping sooner...

HTML5 to the rescue

```
<script src="//other-domain.com/1.js" async></script>  
<script src="2.js" async></script>
```

HTML5 gave us a new attribute, “async”, that assumes you're not going to use `document.write`, but doesn't wait until the document has parsed to execute. The browser will download both scripts in parallel and execute them as soon as possible.

Unfortunately, because they're going to execute as soon as possible, “2.js” may execute before “1.js”. This is fine if they're independent, perhaps “1.js” is a tracking script which has nothing to do with “2.js”. But if your “1.js” is a CDN copy of jQuery that “2.js” depends on, your page is going to get coated in errors, like a cluster-bomb in a... I dunno... I've got nothing for this one.

I know what we need, a JavaScript library!

The holy grail is having a set of scripts download immediately without blocking rendering and execute as soon as possible in the order they were added. Unfortunately HTML hates you and won't let you do that.

The problem was tackled by JavaScript in a few flavours. Some required you to make changes to your JavaScript, wrapping it in a callback that the library calls in the correct order (eg [RequireJS](http://requirejs.org/) (<http://requirejs.org/>)). Others would use XHR to download in parallel then `eval()` in the correct order, which didn't work for scripts on another domain unless they had a [CORS header](https://developer.mozilla.org/docs/HTTP/Access_control_CORS) (https://developer.mozilla.org/docs/HTTP/Access_control_CORS) and the browser supported it. Some even used super-magic hacks, like [LabJS](http://labjs.com/) (<http://labjs.com/>).

The hacks involved tricking the browser into downloading the resource in a way that would trigger an event on completion, but avoid executing it. In LabJS, the script would be added with an incorrect mime type, eg `<script type="script/cache" src="...">`. Once all scripts had downloaded, they'd be added again with a correct type, hoping the browser would get them straight from the cache and execute them immediately, in order. This depended on convenient but unspecified behaviour and broke when HTML5 declared browsers shouldn't download scripts with an unrecognised type. Worth noting that LabJS adapted to these changes and now uses a combination of the methods in this article.

However, script loaders have a performance problem of their own, you have to wait for the library's JavaScript to download and parse before any of scripts it manages can begin downloading. Also, how are we going to load the script loader? How are we going to load the script that tells the script loader what to load? Who watches the Watchmen? Why am I naked? These are all difficult questions.

Basically, if you have to download an extra script file before even thinking about downloading other scripts, you've lost the performance battle right there.

The DOM to the rescue

The answer is actually in the HTML5 spec, although it's hidden away at the bottom of the script-loading section.

Note: The async IDL attribute controls whether the element will execute asynchronously or not. If the element's "force-async" flag is set, then, on getting, the async IDL attribute must return true, and on setting, the "force-async" flag must first be unset...

Let's translate that into "Earthling":

```
[
  '//other-domain.com/1.js',
  '2.js'
].forEach(function(src) {
  var script = document.createElement('script');
  script.src = src;
  document.head.appendChild(script);
});
```

Scripts that are dynamically created and added to the document are async by default, they don't block rendering and execute as soon as they download, meaning they could come out in the wrong order. However, we can explicitly mark them as not async:

```
[  
  '//other-domain.com/1.js',  
  '2.js'  
].forEach(function(src) {  
  var script = document.createElement('script');  
  script.src = src;  
  script.async = false;  
  document.head.appendChild(script);  
});
```

This gives our scripts a mix of behaviour that can't be achieved with plain HTML. By being **explicitly** not async, scripts are added to an execution queue, the same queue they're added to in our first plain-HTML example. However, by being dynamically created, they're executed outside of document parsing, so rendering isn't blocked while they're downloaded (don't confuse not-async script loading with sync XHR, which is never a good thing).

The script above should be included inline in the head of pages, queueing script downloads as soon as possible without disrupting progressive rendering, and executes as soon as possible in the order you specified. "2.js" is free to download before "1.js", but it won't be executed until "1.js" has either successfully downloaded and executed, or fails to do either. Hurrah! async-download but ordered-execution!

Loading scripts this way is supported by everything that supports the async attribute (<http://caniuse.com/#search=async>), with the exception of Safari 5.0 (5.1 is fine). Additionally, all versions of Firefox and Opera are supported as versions that don't support the async attribute conveniently execute dynamically-added scripts in the order they're added to the document anyway.

That's the fastest way to load scripts right? Right?

Well, if you're dynamically deciding which scripts to load, yes, otherwise, perhaps not. With the example above the browser has to parse and execute script to discover which scripts to download. This hides your scripts from preload scanners. Browsers use these scanners to discover resources on pages you're likely to visit next, or discover page resources while the parser is blocked by another resource.

We can add discoverability back in by putting this in the head of the document:

```
<link rel="subresource" href="//other-domain.com/1.js">
<link rel="subresource" href="2.js">
```

This tells the browser the page needs 1.js and 2.js. `link[rel=subresource]` is similar to `link[rel=prefetch]`, but with different semantics (<http://www.chromium.org/spdy/link-headers-and-server-hint/link-rel-subresource>). Unfortunately it's currently only supported in Chrome, and you have to declare which scripts to load twice, once via link elements, and again in your script.

Correction: I originally stated these were picked up by the preload scanner, they're not, they're picked up by the regular parser. However, preload scanner **could** pick these up, it just doesn't yet, whereas scripts included by executable code can never be preloaded. Thanks to Yoav Weiss (<https://twitter.com/yoavweiss>) who corrected me in the comments.

I find this article depressing

The situation is depressing and you should feel depressed. There's no non-repetitive yet declarative way to download scripts quickly and asynchronously while controlling the execution order. With HTTP2/SPDY you can reduce the request overhead to the point where delivering scripts in multiple small individually-cacheable files can be the fastest way. Imagine:

```
<script src="dependencies.js"></script>
<script src="enhancement-1.js"></script>
<script src="enhancement-2.js"></script>
<script src="enhancement-3.js"></script>
...
<script src="enhancement-10.js"></script>
```

Each enhancement script deals with a particular page component, but requires utility functions in dependencies.js. Ideally we want to download all asynchronously, then execute the enhancement scripts as soon as possible, in any order, but after dependencies.js. It's progressive progressive enhancement! Unfortunately there's no declarative way to achieve this unless the scripts themselves are modified to track the loading state of dependencies.js. Even `async=false` doesn't solve this issue, as execution of enhancement-10.js will block on 1-9. In fact, there's only one browser that makes this possible without hacks...

IE has an idea!

IE loads scripts differently to other browsers.

```
var script = document.createElement('script');
script.src = 'whatever.js';
```

IE starts downloading “whatever.js” now, other browsers don’t start downloading until the script has been added to the document. IE also has an event, “readystatechange”, and property, “readyState”, which tell us the loading progress. This is actually really useful, as it lets us control the loading and executing of scripts independently.

```
var script = document.createElement('script');

script.onreadystatechange = function() {
  if (script.readyState == 'loaded') {
    // Our script has download, but hasn't executed.
    // It won't execute until we do:
    document.body.appendChild(script);
  }
};

script.src = 'whatever.js';
```

We can build complex dependency models by choosing when to add scripts into the document. IE has supported this model since version 6. Pretty interesting, but it still suffers from the same preloader discoverability issue as `async=false`.

Enough! How should I load scripts?

Ok ok. If you want to load scripts in a way that doesn't block rendering, doesn't involve repetition, and has excellent browser support, here's what I propose:

```
<script src="//other-domain.com/1.js"></script>
<script src="2.js"></script>
```

That. At the end of the body element. Yes, being a web developer is much like being King Sisyphus (boom! 100 hipster points for Greek mythology reference!). Limitations in HTML and browsers prevent us doing much better.

I'm hoping JavaScript modules (<http://wiki.ecmascript.org/doku.php?id=harmony:modules>) will save us by providing a declarative non-blocking way to load scripts and give control over execution order, although this requires scripts to be written as modules.

Eww, there must be something better we can use now?

Fair enough, for bonus points, if you want to get really aggressive about performance, and don't mind a bit of complexity and repetition, you can combine a few of the tricks above.

First up, we add the subresource declaration, for preloaders:

```
<link rel="subresource" href="//other-domain.com/1.js">
<link rel="subresource" href="2.js">
```

Then, inline in the head of the document, we load our scripts with JavaScript, using `async=false`, falling back to IE's readystate-based script loading, falling back to defer.

```
var scripts = [
  '1.js',
  '2.js'
];
var src;
var script;
var pendingScripts = [];
var firstScript = document.scripts[0];

// Watch scripts load in IE
function stateChange() {
  // Execute as many scripts in order as we can
  var pendingScript;
  while (pendingScripts[0] && pendingScripts[0].readyState == 'loaded') {
    pendingScript = pendingScripts.shift();
    // avoid future loading events from this script (eg, if src changes)
    pendingScript.onreadystatechange = null;
    // can't just appendChild, old IE bug if element isn't closed
    firstScript.parentNode.insertBefore(pendingScript, firstScript);
  }
}
```

```
}

// loop through our script urls
while (src = scripts.shift()) {
  if ('async' in firstScript) { // modern browsers
    script = document.createElement('script');
    script.async = false;
    script.src = src;
    document.head.appendChild(script);
  }
  else if (firstScript.readyState) { // IE<10
    // create a script and add it to our todo pile
    script = document.createElement('script');
    pendingScripts.push(script);
    // listen for state changes
    script.onreadystatechange = stateChange;
    // must set src AFTER adding onreadystatechange listener
    // else we'll miss the loaded event for cached scripts
    script.src = src;
  }
  else { // fall back to defer
    document.write('<script src="' + src + '" defer></'+ 'script>');
  }
}
```

A few tricks and minification later, it's 362 bytes + your script URLs:

```
!function(e,t,r){function n(){for(;d[0]&&"loaded"==d[0][f];)c=d.shift(),c[o]=!i.parentNode.insertBefore(c,i)}for(var
```

```
    "//other-domain.com/1.js",  
    "2.js"  
  ])
```

Is it worth the extra bytes compared to a simple script include? If you're already using JavaScript to conditionally load scripts, as the BBC do (<http://responsivenews.co.uk/post/18948466399/cutting-the-mustard>), you may as well benefit from triggering those downloads earlier. Otherwise, perhaps not, stick with the simple end-of-body method.

Phew, now I know why the WHATWG script loading section is so vast. I need a drink.

Quick reference

Plain script elements

```
<script src="//other-domain.com/1.js"></script>  
<script src="2.js"></script>
```

Spec says: Download together, execute in order after any pending CSS, block rendering until complete. **Browsers say:** Yes sir!

Defer

```
<script src="//other-domain.com/1.js" defer></script>  
<script src="2.js" defer></script>
```

Spec says: Download together, execute in order just before DOMContentLoaded. Ignore “defer” on scripts without “src”. **IE < 10 says:** I might execute 2.js halfway through the execution of 1.js. Isn’t that fun? **The browsers in red** (<http://caniuse.com/#search=defer>) **say:** I have no idea what this “defer” thing is, I’m going to load the scripts as if it weren’t there. **Other browsers say:** Ok, but I might not ignore “defer” on scripts without “src”.

Async

```
<script src="//other-domain.com/1.js" async></script>  
<script src="2.js" async></script>
```

Spec says: Download together, execute in whatever order they download in. **The browsers in red** (<http://caniuse.com/#search=async>) **say:** What’s ‘async’? I’m going to load the scripts as if it weren’t there. **Other browsers say:** Yeah, ok.

Async false

[


```
'1.js',  
'2.js'  
].forEach(function(src) {  
  var script = document.createElement('script');  
  script.src = src;  
  script.async = false;  
  document.head.appendChild(script);  
});
```

Spec says: Download together, execute in order as soon as all download. **Firefox < 3.6, Opera says:** I have no idea what this “async” thing is, but it just so happens I execute scripts added via JS in the order they’re added. **Safari 5.0 says:** I understand “async”, but don’t understand setting it to “false” with JS. I’ll execute your scripts as soon as they land, in whatever order. **IE < 10 says:** No idea about “async”, but there is a workaround using “onreadystatechange”. **Other browsers in red** (<http://caniuse.com/#search=async>) **say:** I don’t understand this “async” thing, I’ll execute your scripts as soon as they land, in whatever order. **Everything else says:** I’m your friend, we’re going to do this by the book.

Except as otherwise noted, the content of this page is licensed under the [Creative Commons Attribution 4.0 License](https://creativecommons.org/licenses/by/4.0/) (<https://creativecommons.org/licenses/by/4.0/>), and code samples are licensed under the [Apache 2.0 License](https://www.apache.org/licenses/LICENSE-2.0) (<https://www.apache.org/licenses/LICENSE-2.0>). For details, see the [Google Developers Site Policies](https://developers.google.com/site-policies) (<https://developers.google.com/site-policies>). Java is a registered trademark of Oracle and/or its affiliates.

Last updated 2013-06-05 UTC.